

```
WiFi CIC-IDS2018 Technical Report@page { size: A4; margin: 1.5cm; } body { font-family:
"Liberation Serif", "DejaVu Serif", serif; color: #111; line-height: 1.45; font-size:
11pt; } h1, h2, h3 { color: #102a43; margin-top: 1.2em; margin-bottom: 0.4em; } h1 {
font-size: 22pt; border-bottom: 2px solid #243b53; padding-bottom: 8px; } h2 {
font-size: 16pt; border-bottom: 1px solid #bccccdc; padding-bottom: 4px; } h3 {
font-size: 13pt; } p { margin: 0.45em 0; text-align: justify; } ul { margin: 0.35em 0
0.6em 1.2em; } li { margin: 0.2em 0; } code { background: #f0f4f8; padding: 0.12em
0.25em; border-radius: 3px; font-family: "Liberation Mono", monospace; font-size: 10pt;
} pre { background: #f8fafc; border: 1px solid #d9e2ec; padding: 10px; white-space:
pre-wrap; font-family: "Liberation Mono", monospace; font-size: 9.5pt; line-height:
1.35; } table { border-collapse: collapse; width: 100%; margin: 0.6em 0 1em 0;
font-size: 10pt; } th, td { border: 1px solid #cbd2d9; padding: 6px 8px; vertical-align:
top; } th { background: #eaf2f8; text-align: left; } .note { background: #fffbea;
border-left: 4px solid #f0b429; padding: 10px 12px; margin: 0.8em 0; } .ok { background:
#f0fff4; border-left: 4px solid #38a169; padding: 10px 12px; margin: 0.8em 0; }
.page-break { page-break-before: always; }
```

Technical Report: CIC-IDS2018 Tuesday Wi-Fi / Network-Flow Pipeline in FL_RANSOM_LLM

Project root:/home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM

Experiment:WiFi_CICIDS2018_Tuesday

Objective of this report:document, in a deep and complete manner, the dataset onboarding, source-code modifications, performance optimizations, execution workflow, final configuration, generated artifacts, and the results obtained in the final experiment.

1. Executive Summary

This work adapted theFL_RANSOM_LLMrepository so that it could ingest and process a large public CIC-IDS2018 CSV directly fromdata/wifi/raw/, transform it into an anomaly-detection-as-language-modeling dataset, distribute benign training shards across federated clients, train a LoRA-adapted SmolLM model, and evaluate the resulting checkpoints on a balanced held-out set.

The refactor replaced the previous toy Wi-Fi logic, which was tied to two miniature CSV files and synthetic fallback data, with a robust single-CSV processor capable of detecting labels, preserving metadata such asTimestampandSrc IP, capping the effective experiment size without touching the raw file, and preparing reproducible train/test/tokenized artifacts.

Additional engineering changes were applied to improve speed and server friendliness: conservative multi-GPU parallel training, DataLoader worker controls, evaluation in batches rather than one sample at a time, and delayed construction ofContentso that textual conversion is performed only for the subset of flows effectively used in the run.

Final experimental footprint.Raw dataset on disk: 3.8G

/home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM/data/wifi/rawProcessed dataset on disk: 589M /home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM/data/wifi/processedResults directory on disk: 226M

/home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM/results/WiFi_CICIDS2018_TuesdayFinal split sizes: train = 250,000 benign flows; test = 250,000 flows (125,000 benign + 125,000 anomalous).

2. Repository and Artifact Inventory

The following file structure is the effective layout used by the final pipeline:

```

FL_RANSOM_LLM/
??? configs/config_wifi.yaml
??? data/wifi/raw/Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv
??? data/wifi/processed/train.csv
??? data/wifi/processed/test.csv
??? data/wifi/processed/tokenized/
??? results/WiFi_CICIDS2018_Tuesday/
?   ??? client_data/
?   ??? communication_metrics.csv
?   ??? f1_scores.csv
?   ??? temporal_metrics.csv
?   ??? round_0 ... round_5/
??? src/data_processing/wifi_processor.py
??? src/federated_learning/client.py
??? src/federated_learning/server.py
??? src/evaluation/evaluator.py

```

3. Dataset Acquisition and Storage Strategy

The selected dataset is `Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv`, from CIC-IDS2018 processed traffic data for machine-learning algorithms. The operational decision was to store the untouched dataset in `data/wifi/raw/` and to ensure that all derived artifacts are created under `data/wifi/processed/`. This preserves reproducibility, keeps the raw source immutable, and makes it possible to reprocess the experiment at any time from the same original file.

The download command used for the server, because the `aws CLI` was unavailable, was:

```

wget -O data/wifi/raw/Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv
"https://cse-cic-ids2018.s3.amazonaws.com/Processed%20Traffic%20Data%20for%20ML%20Algorithms

```

The raw file size observed on the server was approximately 3.8 GB. The first-line schema validation confirmed 84 columns and the presence of the fields required by the new processor, including `Label`, `Timestamp`, `Src IP`, `Dst IP`, `Protocol`, `Flow Bytes`, and `Flow Duration`.

4. Original Wi-Fi Implementation Before the Refactor

Before the modifications described in this report, the Wi-Fi path in the repository was not prepared for the CIC-IDS2018 Tuesday CSV. The pre-refactor processor expected two small files named `benign_wifi.csv` and `anomaly_wifi.csv` in `data/wifi/raw/`, assumed a very small tabular schema composed of `rssi`, `snr`, `latency`, and `throughput`, and could fabricate fake rows if the files were missing or empty.

That old behavior was useful only as a placeholder. It was not suitable for the real CIC-IDS2018 schema, could not read a single large CSV, did not preserve operational metadata needed by the new evaluator, and did not provide a mechanism to cap experiment size cleanly without editing the raw dataset itself.

5. Detailed Code Changes

5.1 `configs/config_wifi.yaml`

A dedicated configuration file for the Wi-Fi/CIC-IDS2018 experiment was created. It defines the dataset identity, raw CSV name, content columns to expose in the text representation, the benign-only training strategy, caps used for the current experiment, federated-learning hyperparameters, and the performance-related options introduced later

in the process.

simulation_name: "WiFi_CICIDS2018_Tuesday"

dataset_name: "wifi"

description: "WiFi/network-flow anomaly detection using the CIC-IDS2018 Tuesday flow CSV."

data_base_path: "./data/"

results_path: "./results/"

Raw dataset placement:

data/wifi/raw/Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv

raw_csv_file: "Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv"

label_column: "Label"

normal_label_values: ["Benign"]

Flow-to-text conversion:

The processor will use whichever of these columns are actually present in the CSV.

wifi_content_columns:

- "Protocol"
- "Flow Duration"
- "Tot Fwd Pkts"
- "Tot Bwd Pkts"
- "TotLen Fwd Pkts"
- "TotLen Bwd Pkts"
- "Flow Byts/s"
- "Flow Bytes/s"
- "Flow Pkts/s"
- "Flow IAT Mean"
- "Fwd IAT Mean"
- "Bwd IAT Mean"
- "Pkt Len Mean"
- "Pkt Len Var"
- "SYN Flag Cnt"
- "ACK Flag Cnt"
- "PSH Flag Cnt"

Split strategy:

Only benign traffic is used for training.

train_benign_fraction: 0.8

benign_calibration_fraction: 0.0

random_seed: 42

test_balance: false

Quick-test controls:

- raw_nrows reads only the first N rows of the raw CSV (fastest, but less representative).

- *_cap limits the size of each processed split without changing the raw dataset.

For full runs, keep these as null.

raw_nrows: null

benign_train_cap: 250000

calibration_benign_cap: null

test_benign_cap: 125000

test_anomaly_cap: 125000

model_name: "HuggingFaceTB/SmolLM-135M"

```

lora: true
lora_rank: 8
lora_alpha_multiplier: 2
lora_dropout: 0.05

use_parallel_training: true
max_parallel_gpus: 2

num_rounds: 5
num_clients: 2
client_frac: 1.0

max_steps: 20
batch_size: 2
dataloader_num_workers: 4
dataloader_pin_memory: true
dataloader_persistent_workers: true
initial_lr: 0.001
lr_scheduler_type: "constant"
warmup_steps: 0

force_reprocess_data: true
evaluator_version: "new"
eval_batch_size: 4
eval_use_autocast: true
eval_torch_dtype: "float16"

top_k_values: [1, 5]
threshold_metric: "top1"
threshold_strategy: "f1_max"
f1_threshold_steps: 101

```

The most relevant configuration decisions are:

- `raw_csv_filepoints` explicitly to the CIC-IDS2018 Tuesday CSV so that legacy toy files do not interfere.
- `benign_train_cap`, `test_benign_cap`, and `test_anomaly_cap` make it possible to run a smaller controlled experiment without touching the raw file.
- `use_parallel_training` and `max_parallel_gpus` control conservative multi-GPU training.
- `dataloader_num_workers` and related flags tune CPU usage without overloading a shared server.
- `eval_batch_size`, `eval_use_autocast`, and `eval_torch_dtype` accelerate the evaluation phase.

5.2 `src/data_processing/wifi_processor.py`

The Wi-Fi processor was completely redesigned. The new version can read a single large CSV, detect the configured label column, normalize labels into binary anomaly targets, select relevant flow features for textual rendering, preserve metadata such as Flow ID, Src IP, Dst IP, and Timestamp, and export the final train/test splits and tokenized dataset in the format expected by the rest of the framework.

The main functional improvements are listed below:

- Single-file loading. The processor resolves `raw_csv_file` from the config and supports the CIC-IDS2018 CSV directly.
- Backward compatibility. The legacy pair `benign_wifi.csv`+`anomaly_wifi.csv` is still accepted as a fallback mode.
- Robust label normalization. Text labels such as `Benign` and numeric `0/1` values are

normalized into a clean binary `columnLabel`, while the raw original label is preserved in `label_raw`.

- Flexible feature selection. The processor uses whichever of the configured preferred columns actually exist in the raw schema.
- Benign-only training strategy. Only benign flows are written to `train.csv`; test data is composed of benign holdout plus anomalous flows.
- Caps without raw-data mutation. The current run uses 250k benign training flows and a balanced 250k test set via configuration only.
- Metadata preservation. The final CSVs keep fields that later enable flow-level and temporal evaluation.
- Text generation after capping. This optimization is crucial: Content is now created only for the rows that survive the configured caps, instead of being created for millions of discarded rows.
- Sanitization. IP and MAC-like patterns are masked inside Content while preserving numeric flow metrics.
- Tokenizer preparation. The processed benign training split is turned into a Hugging Face `Dataset/DatasetDict` structure and saved under `processed/tokenized/`.

5.3 `src/federated_learning/server.py`

The server already contained a sequential and a parallel path, but the experiment-specific configuration was not enabling the parallel branch. The final configuration now activates `use_parallel_training`, and the server code was refined so that the number of GPUs used can be capped explicitly via `max_parallel_gpus`.

This change is important on a shared server because it allows controlled use of hardware: the experiment can use two GPUs when appropriate, while still respecting a project-level limit and avoiding accidental uncontrolled fan-out.

5.4 `src/federated_learning/client.py`

The client-side training code was augmented with conservative `DataLoader` controls. Specifically, the training arguments now accept `data_loader_num_workers`, `data_loader_pin_memory`, and `data_loader_persistent_workers` when the installed `transformers` version supports them. For the final server-friendly configuration, this was set to four workers per process.

The intent was not to saturate all 32 CPU threads on the machine, but to reduce starvation of the GPUs and data-delivery stalls while keeping the experiment respectful of a shared academic environment.

5.5 `src/evaluation/evaluator.py`

The evaluator received the most important speed optimization. In the original evaluation logic, the code tokenized and scored one text sample at a time in a serial loop. For a test set with 250,000 flows, this would still be very slow and underutilize the GPU. The new evaluator uses batched tokenization and batched inference, with optional autocast in half precision. This change substantially reduces the runtime of the evaluation phase and makes GPU usage much more meaningful.

The evaluator now also clears model objects and CUDA cache between rounds more explicitly, which helps keep memory usage stable while iterating over all checkpoints.

6. End-to-End Pipeline Description

6.1 Entry point and orchestration

The process is orchestrated by `main.py`. The runtime first loads the YAML configuration,

applies Hugging Face cache/offline environment options, instantiates the dataset-specific processor based on `dataset_name`, runs preprocessing/tokenization, launches federated training, and then invokes the selected evaluator.

For this experiment, the control flow is:

- `main.py` loads `configs/config_wifi.yaml`.
- `WiFiProcessor` creates `train.csv`, `test.csv`, and `tokenized/`.
- `FederatedServer` initializes the global model and shards the benign tokenized training set.
- Two federated clients are trained across five rounds.
- `NewEvaluator` reevaluates checkpoints from rounds 1 through 5 on the balanced test set.

6.2 Train/test generation for the final experiment

The final experiment uses the following logic:

- All benign rows are shuffled and split into a training portion and a benign holdout portion.
- The benign training set is capped to 250,000 rows.
- The benign holdout is capped to 125,000 rows.
- The anomaly subset is capped to 125,000 rows.
- The final test set is the shuffled union of the benign holdout and anomaly cap.

This preserves the anomaly-detection philosophy of the repository: learning is performed on normal behavior only, and the model is later evaluated on a mixture of benign and malicious flows.

6.3 Flow-to-text representation

Each selected network flow is converted into a textual sequence in the `Content` field. The generated tokens correspond to semantically meaningful flow features such as protocol, duration, packet counts, byte rates, inter-arrival times, packet length statistics, and flag counters. The goal is to let the language model learn regularities over a textualized representation of network behavior rather than over raw numeric tensors only.

The chosen features for the final run were:

- Protocol
- Flow Duration
- Tot Fwd Pkts
- Tot Bwd Pkts
- TotLen Fwd Pkts
- TotLen Bwd Pkts
- Flow Byts/s
- Flow Pkts/s
- Flow IAT Mean
- Fwd IAT Mean
- Bwd IAT Mean
- Pkt Len Mean
- Pkt Len Var
- SYN Flag Cnt
- ACK Flag Cnt
- PSH Flag Cnt

6.4 Tokenization

The processed benign training subset is loaded from train.csv, converted to a Hugging Face dataset, tokenized with the SmolLM tokenizer, and stored in data/wifi/processed/tokenized/. Only benign training data is tokenized for the federated learning stage, because the approach treats anomaly detection as modeling normality.

6.5 Client sharding

The tokenized benign dataset is then split IID across two clients. The resulting metadata is:

Client	Samples
0	125000
1	125000

Each client therefore received exactly 125,000 benign training examples.

7. Execution Environment and Server Constraints

The experiment ran on a shared academic server, so performance tuning had to balance speed and etiquette. The hardware snapshot captured during the analysis phase is reproduced below.

7.1 Host information

```
avancos\nLinux avancos 6.17.0-19-generic #19-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar 6  
14:02:58 UTC 2026 x86_64 GNU/Linux
```

7.2 CPU and memory

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Address sizes:	46 bits physical, 48 bits virtual
Byte Order:	Little Endian
CPU(s):	32
On-line CPU(s) list:	0-31
Vendor ID:	GenuineIntel
Model name:	13th Gen Intel(R) Core(TM) i9-13900K
CPU family:	6
Model:	183
Thread(s) per core:	2
Core(s) per socket:	24
Socket(s):	1
Stepping:	1
CPU(s) scaling MHz:	63%
CPU max MHz:	5400.0000
CPU min MHz:	800.0000
BogoMIPS:	5990.40
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfperf tsc_known_freq pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb intel_pt sha_ni xsaveopt xsavec xgetbv1 xsaves split_lock_detect user_shstk avx_vnni dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp hwp_pkg_req hfi vnmi umip pku ospke

```
waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize pconfig
arch_lbr ibt flush_l1d arch_capabilities
Virtualization: VT-x
L1d cache: 896 KiB (24 instances)
L1i cache: 1.3 MiB (24 instances)
L2 cache: 32 MiB (12 instances)
L3 cache: 36 MiB (1 instance)
NUMA node(s): 1
NUMA node0 CPU(s): 0-31
Vulnerability Gather data sampling: Not affected
Vulnerability Ghostwrite: Not affected
Vulnerability Indirect target selection: Not affected
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Mmio stale data: Not affected
Vulnerability Old microcode: Not affected
Vulnerability Reg file data sampling: Mitigation; Clear Register File
Vulnerability Retbleed: Not affected
Vulnerability Spec rstack overflow: Not affected
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled
via prctl
Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user
pointer sanitization
Vulnerability Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB
conditional; PBR SB-eIBRS SW sequence; BHI BHI_DIS_S
Vulnerability Srbds: Not affected
Vulnerability Tsa: Not affected
Vulnerability Tsx async abort: Not affected
Vulnerability Vmscape: Mitigation; IBPB before exit to userspace

total      used      free      shared buff/cache   available
Mem:      123Gi      47Gi      6.6Gi      41Mi      70Gi      75Gi
Swap:      30Gi      156Ki      30Gi
```

7.3 GPU inventory

NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver. Make sure that the latest NVIDIA driver is installed and running.

The machine provides an Intel Core i9-13900K with 32 logical CPUs, 123 GiB of RAM, and two NVIDIA RTX 4090 GPUs with 24 GiB each. Monitoring also showed that, prior to the speed-oriented changes, the code underutilized both CPU and GPU resources, often keeping most CPU threads idle while only partially engaging a single GPU.

8. Performance Engineering Decisions

The tuning strategy followed a conservative philosophy appropriate for a multi-user server:

- Use at most two GPUs because the machine physically has two, and because the experiment already uses exactly two clients.
- Limit DataLoader workers to four per process rather than aggressively using all CPUs.
- Keepbatch_size = 2for training to avoid unexpected VRAM pressure.
- Useeval_batch_size = 4with half precision to accelerate evaluation while keeping memory risk low on 24 GiB cards.
- Delay expensive text conversion until after caps are applied to the raw data.Important

operational observation. The parallel training mode improved hardware utilization but did not reduce total training time for this relatively small experiment. The additional process launch and model-loading overhead outweighed the gain in raw compute. However, batched evaluation produced a much more meaningful speedup because the original evaluator was strongly serial.

9. Commands Used to Reproduce the Experiment

The following commands summarize the final operational workflow used on the server:

```
cd ~/Artigo-LANC-Thiago/FL_RANSOM_LLM\nsource venv/bin/activate\nmkdir -p data/wifi/raw
data/wifi/processed\nwget -O
data/wifi/raw/Thursday-20-02-2018_TrafficForML_CICFlowMeter.csv
\"https://cse-cic-ids2018.s3.amazonaws.com/Processed%20Traffic%20Data%20for%20ML%20Algorithm
main.py --config configs/config_wifi.yaml
```

10. Training Observations

The training stage ran for five rounds, using two clients and LoRA rank 8 on SmolLM-135M. The observed per-round training logs from the final run are summarized below.

Round	Client 0 Loss	Client 1 Loss	Round Time (s)	Observation
1	1.9170	1.9170	25	Parallel training initialized and both GPUs were engaged.
2	0.8371	0.8640	22	Fast loss drop indicates that the model quickly learned the benign-flow language pattern.
3	0.6368	0.6999	21	Training entered a more stable region with diminishing gains.
4	0.6569	0.6474	21	Small oscillation, still within the same stability band.
5	0.5925	0.6257	22	Lowest observed training losses, but evaluation did not improve.

Loss dropped sharply between rounds 1 and 2 and then stabilized. This is a useful signal that the model learned the benign-flow language pattern quickly. Nevertheless, training loss alone is not enough; the downstream anomaly-detection metrics must also improve in a balanced way.

11. Communication Metrics

The federated server saved communication-cost information for every round:

Round	Selected Clients	Bytes Total	Bytes Mean / Client	Params Total
1	2	3686400	1843200.0	921600
2	2	3686400	1843200.0	921600
3	2	3686400	1843200.0	921600
4	2	3686400	1843200.0	921600
5	2	3686400	1843200.0	921600

Every round transmitted the same update volume: 3,686,400 bytes in total and 921,600 LoRA parameters across the two clients. This stability is expected because the model architecture and LoRA rank remain constant throughout the run.

12. Evaluation Results

12.1 Best checkpoints by top-k setting

K	Best Round	F1	Precision	Recall	Benign FPR	Threshold
1	2	0.6949	0.5352	0.9907	0.8604	0.88
5	2	0.6751	0.5143	0.9821	0.9275	0.97

The best overall result in the final experiment was obtained for $K = 1$ at round 2, with $F1 =$

0.6949.

12.2 Full F1 table

	Round	K	Threshold	F1	Precision	Recall	Benign FPR
	1	1	0.76	0.6753	0.5122	0.9907	0.9436
	1	5	0.92	0.6703	0.5053	0.9951	0.9741
	2	1	0.88	0.6949	0.5352	0.9907	0.8604
	2	5	0.97	0.6751	0.5143	0.9821	0.9275
	3	1	0.89	0.6941	0.5342	0.9904	0.8635
	3	5	0.98	0.6725	0.5083	0.9935	0.9611
	4	1	0.92	0.6833	0.5198	0.9967	0.9207
	4	5	0.98	0.6745	0.5100	0.9956	0.9565
	5	1	0.92	0.6938	0.5315	0.9988	0.8803
	5	5	0.98	0.6740	0.5097	0.9945	0.9565

12.3 Temporal metrics

	Round	K	Mean TTD (s)	Median TTD (s)	Coverage	Benign FPR	Attacked Devices
	1	1	0.0000	0.0000	1.0000	0.9436	10
	1	5	0.0000	0.0000	1.0000	0.9741	10
	2	1	0.0000	0.0000	1.0000	0.8604	10
	2	5	0.0000	0.0000	1.0000	0.9275	10
	3	1	0.0000	0.0000	1.0000	0.8635	10
	3	5	0.0000	0.0000	1.0000	0.9611	10
	4	1	0.0000	0.0000	1.0000	0.9207	10
	4	5	0.0000	0.0000	1.0000	0.9565	10
	5	1	0.0000	0.0000	1.0000	0.8803	10
	5	5	0.0000	0.0000	1.0000	0.9565	10

The temporal table shows zero time-to-detection and full coverage for all rounds because the selected thresholds cause nearly immediate detection once anomalous flows appear. However, this apparently perfect temporal responsiveness is offset by an unacceptably high benign false-positive rate, so it should not be interpreted as operational readiness.

13. Interpretation of the Final Results

The final experiment demonstrates that the pipeline is technically functional and that the model does learn a separation signal. F1 values around 0.69 on a balanced test set confirm that the system is not random. The best checkpoint emerged early, around round 2, which means this setup converges quickly under the current training budget.

However, the model's operational profile is not yet satisfactory. The most striking issue is the extremely high benign false-positive rate. For example, the best overall K=1 result at round 2 achieved F1=0.6949 and recall=0.9907, but also a benign FPR=0.8604. In plain terms, the system catches almost all anomalous traffic, but it misclassifies a very large fraction of normal traffic as anomalous.

This means the current setup is useful as a proof-of-pipeline and as an experimental baseline, but not yet as a production-quality anomaly detector. The most likely next improvements would be threshold calibration on a benign-only calibration set, alternative feature engineering forContent, larger or more representative training subsets, and more operational threshold selection such as target-FPR tuning.

14. Files Added or Modified During This Work

- configs/config_wifi.yaml: created and iteratively refined to control dataset loading,

- caps, federated settings, and performance parameters.
- src/data_processing/wifi_processor.py: replaced placeholder toy logic with a full CIC-IDS2018 pipeline.
- src/federated_learning/server.py: refined parallel GPU selection through `throughmax_parallel_gpus`.
- src/federated_learning/client.py: added DataLoader worker configuration hooks to the training arguments.
- src/evaluation/evaluator.py: changed evaluation from serial single-sample inference to batched inference with autocast and explicit CUDA cleanup.
- docs/generate_wifi_report.py: added to generate this report reproducibly.

15. Practical Guidance for Future Runs

- If the objective is quick iteration, keep the current caps.
- If the objective is higher fidelity, raise `benign_train_cap` and possibly `test_*_cap`, but be aware that preprocessing, tokenization, and evaluation time will grow.
- If VRAM becomes a concern, reduce `eval_batch_size` from 4 to 2.
- If the server becomes busier with other students, reduce `data_loader_num_workers` from 4 to 2.
- For scientific evaluation quality, add a benign calibration split and use operational thresholding rather than only `oracle_f1_max`.

16. How to Download the PDF

The generated PDF is written to:

`/home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM/docs/WiFi_CICIDS2018_Technical_Report.pdf`

It can be copied to another machine with a command such as:

```
scp
```

```
avancos:/home/weverton/Artigo-LANC-Thiago/FL_RANSOM_LLM/docs/WiFi_CICIDS2018_Technical_Report.pdf
```

17. Closing Statement

This documentation captures the complete Wi-Fi/CIC-IDS2018 onboarding and experiment lifecycle performed in the repository: dataset download and organization, code refactor, performance tuning, controlled federated training, batch evaluation, and interpretation of the resulting metrics. The repository is now in a materially more reproducible and usable state for future experiments on large network-flow datasets.